

PROJECT REPORT

HOMEWORK 2

Written and Prepared by:

Roxanne Lutz

University of New Mexico

September 19, 2025

COMPUTER/MATLAB

ORIGINAL PROBLEM STATEMENT

“Use Newton’s method to approximate, to within 10^{-4} , the value of x that produces the point on the graph of $y = x^2$ that is closest to $(1, 0)$. Hint: Minimize $[d(x)]^2$, where $d(x)$ represents the distance from (x, x^2) to $(1, 0)$. ”

PART A

“Derive by hand the problem that needs to be solved in the form of $f(x) = 0$. ”

See the below derivation.

Part a Derive by hand the problem that needs to be solved in the form of $f(x) = 0$.

Distance from 2 points equation.

$$d(x_1, x_2, y_1, y_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$[d(x_1, x_2, y_1, y_2)]^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

$$\begin{matrix} (1, 0) \\ (x, x^2) \end{matrix} \rightarrow [d(1, x, 0, x^2)]^2 = (x-1)^2 + x^4$$

To find minimum, we can find when the derivative of the above is equal to zero:

$$d \dots [d(\dots)]^2 = (x-1)^2 + x^4$$

$$\frac{d}{dx} [d(\dots)]^2 = 2(x-1) + 4x^3 dx$$

$$= 2x - 2 + 4x^3$$

$$= 4x^3 + 2x - 2 = f(x)$$

Thus:

$$f(x) = 0 = 4x^3 + 2x - 2$$

PART B

“Review the way the Pseudo-code is given on page 67 (Burden 10ed). They have an output for both success and failed iterations. Your code should be an improved version over what we covered in class and this mentioned Pseudo-code. In other words, polish up your code.”

As follows is the code written in the script file, which is intended to be an improvement upon both pseudocode and in-class examples.

```
% some initial values
tolerance = 10 ^ (-4); % allowed to get approximate within 10^-4
as stated within problem.
iterations = 20; % how many times to iterate
x_curr = 2; % initial guess
% flag for a successful approximation within these parameters
success = false;
% for counting the actual iterations used for easy printing
iterationsNeeded = 0;
for i = 1:iterations;
    iterationsNeeded = iterationsNeeded + 1; % for printing only
    [y, dy] = function1(x_curr); % get f(x) and f'(x)
    % hold values for plotting
    xapprox(i) = x_curr;
    yapprox(i) = y;
    if abs(y) < tolerance; % found 0 within tolerance
        success = true;
        break;
    end;
    x_curr = x_curr - (y ./ dy); % apply newtons method;
end;
% print success if we've found a zero within iterations and
tolerance
if success;
    disp( ...
        "Approximate root found successfully in " ...
        + iterationsNeeded ...
        + " iterations: x = " ...
        + x_curr ...
        );
else;
    disp( ...
```

```

        "Approximate root was NOT found within tolerance in " ...
        + iterations ...
        + " iterations." ...
    );
end;

```

And, for clarity, as follows is our distance equation derived, along with its second derivative, implemented in MATLAB.

```

% this is the function to find roots of
% which is the distance function between
% y = x^2 and point (1, 0)
% f(x) = 4x^3 + 2x - 2
% f'(x) = 12x^2 + 2
function [y, dy] = function1(x)
    y = (4 * (x .^ 3)) + (2 * x) - 2;
    dy = (12 * (x .^ 2)) + 2;
end

```

PART C

“Implement Newton’s method with $x_0 = 2$. How many iterations does it take for the method to converge?”

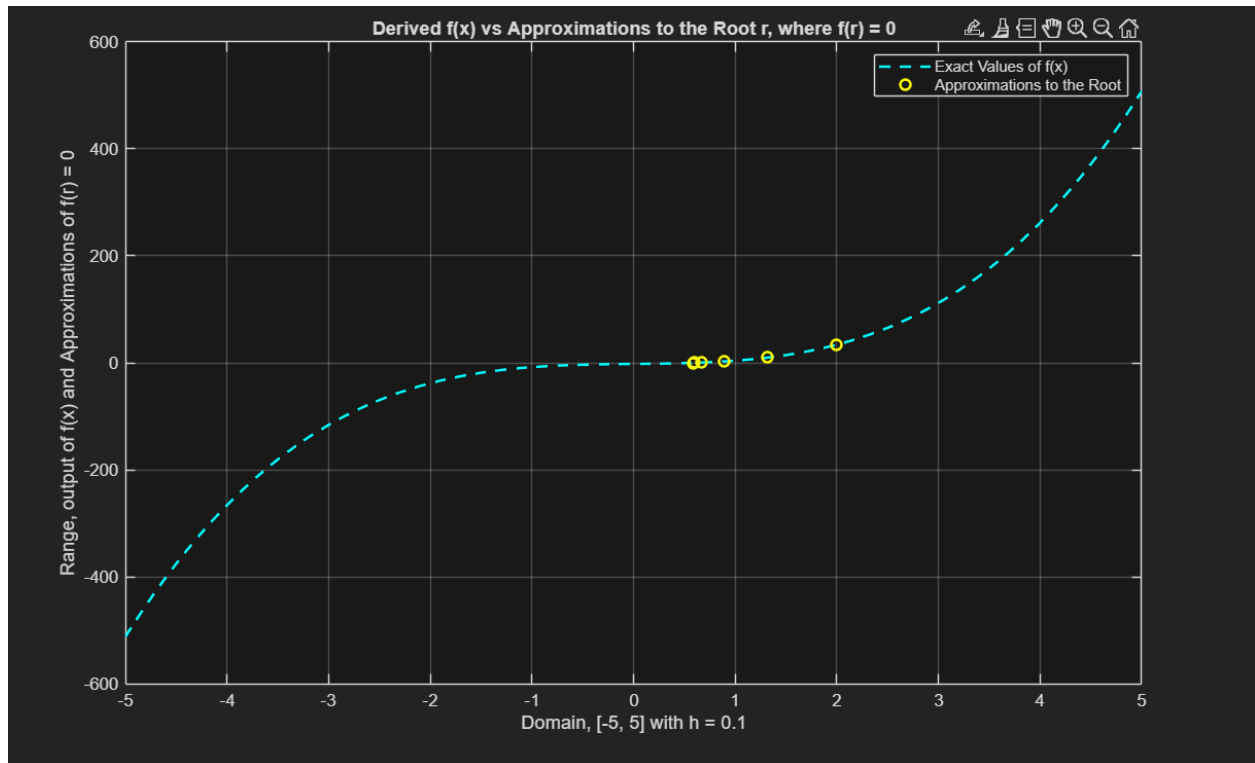
As we run the code above, including the first iteration always, it takes Newton’s method 7 times to converge to a “close-enough” approximation to the exact root of $x \approx 0.58975$.

Note that setting x_0 to 5 takes 9 iterations to find the root, whereas $x_0 = 0.5$ takes only 4 iterations. So, clearly, the closer we are to the actual root, the less iterations we need to get a decent approximation (“decent” as being defined by us.)

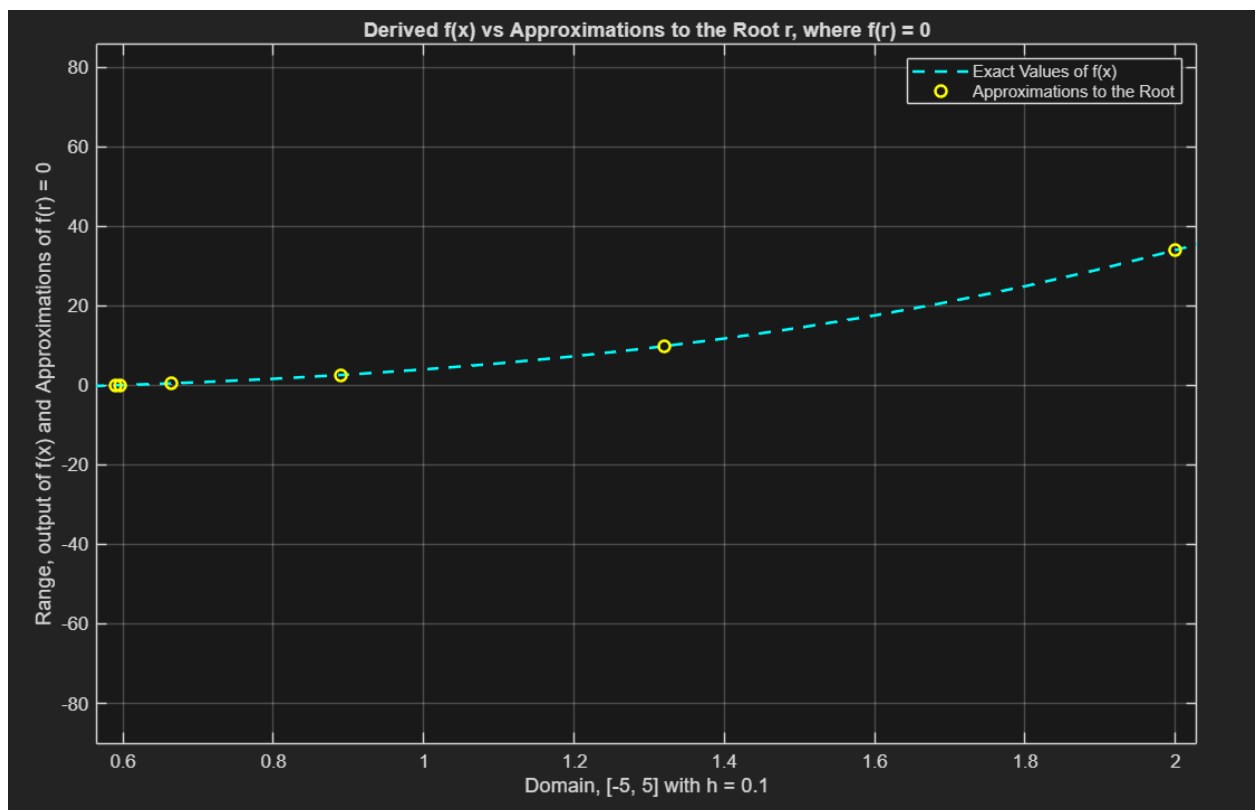
PART D

“To help understand the solution given by Newton’s method, provide a graphical representation of what was found. Describe your findings.”

Here is a simple plotting of $x_0 = 2$ Newton’s method approximations for the root against the exact values of our derived $f(x)$.



To even clearer see the approximation approach, we will zoom in slightly.



From here, there are a few things to note.

It is very evident that as Newton's method progresses closer to the root each time (which means we chose our x_0 well and did not fall into a pitfall of Newton method failure).

Further, the distance between each approximation (as in $x_0 \rightarrow x_1$, $x_2 \rightarrow x_3$, and on until the root is found) decreases with each iteration. The approximation converges quite rapidly to the actual root with each iteration. I would hypothesize that this likely a consequence of the fact that if x_0 is chosen sufficiently close to the actual root r that the approximation converges quadratically to r , or, as indicated mathematically:

$$|r - x_{n+1}| \leq c|r - x_n|^2$$

CONCLUSION

To conclude, Newton's method is a decent way to find the root of an equation. If we can phrase a problem in the context of finding a root of a function, this is a simple scheme to utilize to get a close approximation in a minimal amount of iteration computation.

However, to use this method, we had to have a consistent derivative that was doable by hand in order to use this scheme. In many real-life data, this may not always be so clear and straightforward. The original function, therefore, is a crucial part of the usability of Newton's method.

The method itself also relies on our ability to pick a reasonable initial guess (x_0). There are situations—most of which we have covered in class—that could render the method completely useless, giving nonsensical or impossible results.

Regardless, if we have a correct situation and have the discipline to set up this scheme wisely, Newton's method is an excellent and simple way to achieve the close approximation of a function's root.